ML → AI Engineer Learning Roadmap v3 — Tailored to Your Current Skills (with links)

Starting Aug 11 | ~12 weeks total | 8-9 hrs/day, with built-in rest

Weeks 1-6: ML foundations (math, classical ML, basic deep learning, deployment basics)
Weeks 7-12: AI Engineer specialization (LLMs/RAG, backend APIs, Docker, cloud deployment) — added because "AI Engineer" roles today expect LLM application-building and software engineering skills on top of classical ML, not instead of it.

Why this version is different from v1

- v1 had you relearning Python/pandas/EDA you already know — cut.
- v1 crammed math + deep learning into single weeks with no slack — expanded.
- v1 had zero rest days across 30 days — added 1 buffer/rest day per week.
- v1 had no spaced review — added short recall sessions every few days on *previous* week's material, not just same-day revision.
- v2 stopped at classical ML + basic deep learning — v3 adds the LLM/RAG/backend/deployment layer needed to genuinely apply for "AI Engineer" titles, not just "Junior ML Engineer."

Week 1: Math Intuition + ML Foundations (Aug 11–17)

Skipped: Python/pandas/numpy/EDA basics — you already have this.

- **Math (3-4 days):**
  - 3Blue1Brown — Essence of Linear Algebra (playlist)
  - 3Blue1Brown — Essence of Calculus (playlist)
  - StatQuest — full channel & video index | StatQuest YouTube
- **Intro ML theory (3 days):**
  - Andrew Ng — Machine Learning Specialization (Coursera, audit free) — beginner-friendly, practical; use this as your primary source, not the old grad-level CS229 lectures
  - Kaggle Learn — Intro to Machine Learning (free micro-course) — a few hours, hands-on, great since you already know pandas/EDA
  - Primary channel pick: CampusX — 100 Days of Machine Learning — **CONFIRMED primary source** (Hindi explanations are working well for retention; stick with this throughout)
- **Day 7: Buffer/rest + light review**

Optional deeper pass later: Stanford CS229 full course, Autumn 2018 — grad-level and math-heavy, best as a second pass once the practical basics are solid, not a first introduction.

Mini project: Skip Titanic as a "learning" exercise — instead do a **quick model pass** (logistic regression + decision tree) on a dataset you're already comfortable EDA-ing, just to bridge into modeling. Don't over-invest time here.

Week 2: Classical ML — Core Focus (Aug 18–25, extended to 8 days)

This is your real starting line. No skipping. Extended by 1 day: merge/groupby/join were a genuine gap (not just refresher-level), so Day 8 is fully dedicated to closing that gap before starting the projects. Borrowed from Week 3's buffer day per the "if you fall behind" rule.

Topics: Pandas merge/join/concat/groupby (Day 8), Regression, Classification, Decision Trees, Ensembles (Random Forest, Boosting), Clustering, Dimensionality Reduction, Feature Engineering, Cross-Validation, Hyperparameter Tuning

Day-by-day plan:

Day	Focus
Day 8	Full day: pandas merge, join, concat, groupby — theory + hands-on (toy datasets, then apply to HR data)
Day 9	Linear Regression + Logistic Regression; start HR Retention rebuild
Day 10	Decision Trees; mini-project comparison folded into HR Retention
Day 11	Random Forest, Bagging
Day 12	Boosting (AdaBoost, Gradient Boosting); wrap up HR Retention
Day 13	Clustering + PCA; start House Prices Prediction
Day 14	Feature Engineering + Cross-Validation + Hyperparameter Tuning; Kaggle Learn micro-courses; finalize House Prices
Day 15	Buffer/rest day + self-test

Resources:

- Kaggle Learn — Intermediate Machine Learning (missing values, categorical variables, data leakage — hands-on, a few hours)
- Kaggle Learn — Feature Engineering
- Primary video source: CampusX (confirmed) — only reach for Krish Naik or Codebasics as backup if a specific explanation doesn't click
- **Pandas merge/groupby (Day 8):** search within the CampusX channel for their pandas merge and groupby sessions; code along in Jupyter rather than watching passively. Backup if needed: Corey Schafer — combining DataFrames, Real Python — merge/join/concat guide
- **EDA refresh (Day 9, before HR Retention rebuild):** you already have real EDA experience — skim 1-2 CampusX EDA videos max (20-30 min) as a light refresher, then go straight into applying it. Don't over-invest time re-studying something you already know; reinforce by doing, not by rewatching.
- StatQuest — video index (Random Forest / SVM / Boosting — best for building intuition on *why* an algorithm works, use alongside your primary source rather than instead of it)

Projects:

1. HR Employee Retention — rebuild with proper models + evaluation metrics (precision/recall/F1, not just accuracy), incrementally adding logistic regression → decision tree → random forest → boosting across Days 9-12
2. House Prices Prediction (Kaggle) — Days 13-14

Day 7: Buffer + recall session on Week 1 math (quick self-quiz, no new material)

Week 3-4: Deep Learning — Given Double the Time (Aug 25–Sep 7)

The most compressed part of v1. Now it gets 2 full weeks. Note: Week 3 actually starts the Monday after a buffer day, per real pacing — dates are approximate, follow the day count not the calendar.

Week 3 — finalized day-by-day plan:

Day	Topic	Focus
Day 1	Introduction to Deep Learning + Perceptron	What is DL, Types of NN, Perceptron basics
Day 2	Perceptron Training + Loss Function	Perceptron Trick, Loss Functions, Limitations of Perceptron
Day 3	Multi-Layer Perceptron + Activation Functions	MLP Intuition, Activation Functions (Sigmoid, Tanh, ReLU)
Day 4	Forward Propagation + Backpropagation	How NN predicts + how it learns
Day 5	First ANN Project (Keras)	Customer Churn Prediction or Handwritten Digit Classification
Day 6	Rebuild same project in PyTorch	Framework transfer practice — same problem, new tool, so focus stays on syntax/mental model, not a new dataset too
Day 7	Buffer + Extra Practice	Revision + Graduate Admission Prediction (regression with ANN) if time allows

Resources: CampusX (theory, Days 1-4, primary source) | Keras/TensorFlow (Day 5 build) | Aladdin Persson — YouTube channel (PyTorch reference for Day 6) | deeplizard — YouTube channel (Neural Networks / PyTorch series, backup)

Why Keras first, then PyTorch: Keras' higher-level API lets Day 5 focus purely on ANN concepts without fighting syntax. Day 6 rebuilding the *same* project in PyTorch isolates the framework-learning curve, since the problem itself is already understood — important groundwork for Weeks 7-8, which are PyTorch-centric (LangChain/Hugging Face ecosystem).

Week 4 — revised day-by-day plan (per external roadmap review — 8 days, one evaluated project instead of a second framework rebuild):

Day	Topic	Focus	Framework/Resource
Day 1	Introduction to CNN	What is CNN, why CNN > ANN for images, Convolution operation	CampusX
Day 2	CNN Building Blocks	Padding, Stride, Pooling (Max/Average), Flatten	CampusX
Day 3	CNN Architecture + Backprop in CNN	Full CNN flow + backprop in Conv/Pooling layers	CampusX
Day 4	First CNN Project	Image Classification (Cats vs Dogs or CIFAR-10/Fashion MNIST)	Keras
Day 5	Improve CNN	Dropout, Batch Normalization, Data Augmentation	Keras
Day 6	Transfer Learning + Error Analysis	Compare a pretrained model (e.g. ResNet-18) against your Day 4/5 CNN trained from scratch; inspect misclassified images and write a short explanation of *why* the model got them wrong	Keras or PyTorch, your choice
Day 7	RNN/LSTM — light overview	What RNNs are, why they exist (sequential data), vanishing gradient problem, what LSTM fixes — conceptual + one small guided example. Kept deliberately light: not a direct prerequisite for Transformer-based LLM work in Weeks 7-8	CampusX or deeplizard
Day 8	Buffer + Revision	Revise weak areas, clean notebooks, self-test on the week	—

Revised per external roadmap review: dropped the separate "rebuild in PyTorch" day. You already did one full PyTorch framework-transfer exercise in Week 3 (Churn) — repeating it here for CNN duplicates effort without much new learning. **PyTorch is now your primary deep-learning framework going forward**; use it directly for Day 6's transfer learning comparison if you're comfortable, or Keras if that's faster for this specific comparison — either is fine, since the goal here is the *comparison and error analysis*, not framework practice.

Resources: fast.ai — Practical Deep Learning for Coders (supplementary, especially for CNN/transfer learning if CampusX explanation needs backup)

Project — the actual deliverable for Week 4, not just practice: One evaluated CNN/transfer-learning project, built across Days 4-6:

1. Baseline CNN (Day 4) — examine training/validation curves, not just final accuracy
2. Add Dropout, Batch Normalization, Data Augmentation (Day 5) — record what actually improved validation performance, not just "I added it"
3. Compare against one pretrained model (Day 6) — inspect several misclassified images and write a short explanation of the pattern in what's failing (e.g., "confuses cats and dogs in low-light images," not just "some errors")
4. **Data hygiene, same standard as every project so far:** proper train/val/test split, fit any learned preprocessing (like normalization stats) on training data only, keep random augmentation out of evaluation, check for duplicate/near-duplicate images crossing your splits

Day 8: Buffer + recall on the week

Week 5: Software Engineering Basics + Deploy Your Best Existing Project (Sep 8–14)

Revised per external roadmap review: don't wait until Week 9 to discover whether a notebook works as a real application. Deploy something you already understand deeply (HR/Churn) instead of starting a new project — a new sales-forecasting project would add a separate, unrelated learning problem (chronological validation, forecasting baselines, time-related leakage) at the wrong moment.

Topics:

- Python modules/functions, clean project structure, dependency management (`requirements.txt` or similar)
- Git basics, environment variables, keeping credentials out of your repository
- A small FastAPI endpoint with input validation and useful error messages
- A few real tests: preprocessing, prediction, invalid-input handling

Project: Take your strongest existing ML project — **HR Retention or the Churn ANN** — and turn it into a real, callable service:

1. Refactor the notebook code into clean Python modules/functions (not just notebook cells)
2. Wrap the trained model in a small FastAPI app with a `/predict` endpoint, proper input validation (e.g., reject malformed input with a clear error, not a stack trace)
3. Write a handful of tests — does preprocessing behave correctly on clean input? Does the endpoint reject bad input properly? Does prediction return the expected shape?

4. Use Git properly — meaningful commits, `.gitignore` for anything sensitive, no hardcoded secrets

   Resources:

- FastAPI — Official Tutorial
- Git basics — official docs

  Day 7: Buffer + review

Week 6: Portfolio Basics + Applications Begin (Sep 15–21)

- Clean up all GitHub repos with proper READMEs (problem, approach, results, what you'd improve) — including your new Week 5 deployed service
- Update LinkedIn + resume with real projects and metrics, not just tool lists
- Small, tested FastAPI service from Week 5 becomes a genuine portfolio piece — link it, describe the architecture briefly in the README
- **Kaggle competition: now optional**, not required — only do it if you have spare time after the above; the deployed project + clean portfolio matters more for applications than an extra competition entry
- Start applying to internships from Day 4 onward — don't wait until everything is "perfect"

  Day 7: Buffer + review

Daily Structure (gym-adjusted schedule)

- **9:00 – 12:30 (3.5 hrs):** Structured learning (videos/notes) — heaviest theory block, done fresh
- **12:30 – 2:00:** Lunch break
- **2:00 – 4:00 (2 hrs): Protected coding/implementation block — not more videos.** Building, debugging, testing, evaluating your actual project. On theory-only days early in a topic, this can flex toward applying what you just learned in a small exercise, but default to this being real hands-on work.
- **4:00 – ~6:00:** Gym break
- **Night (2-3 hrs):** Coding/projects continued + 30-60 min LeetCode (Easy-Medium Python) + SQL + 5-10 min daily learning log + **30 min English practice (tenses)** — see dedicated section below
- **Total: ~8-9 hrs/day**

Revised per external roadmap review: the original schedule risked project coding only happening in whatever night-time energy was left after LeetCode/SQL/review. **Target at least half your daily study time actually implementing, debugging, testing, or evaluating — not watching someone else do it.** If a day's video content runs long, protect the 2:00-4:00 coding block first and trim from the morning learning block instead, not the other way around.

If the night coding block consistently feels weak after the gym, that's fine — the 2:00-4:00 block is now the protected one; night can lean more toward LeetCode/SQL/review on those days.

English Practice (30 min/night) — Tenses Focus

Added to fix sentence-structure and spelling mistakes, dual-purposed with the daily log so it doesn't cost extra unrelated time.

- **15 min — Study:** Free Udemy course — "English Grammar in Simple Urdu/Hindi: Tenses" — covers all 12 English tenses, explained in Urdu/Hindi, built for beginners who need a proper foundation. Go one tense lesson at a time.
  - Backup/alternative if preferred: Mastering Tenses — Present + Past + Future (Urdu/Hindi) YouTube playlist
- **15 min — Apply immediately:** rewrite 2-3 sentences from that day's ML daily log using the tense just studied, correctly. Ties grammar practice directly to real sentences already being written, instead of disconnected textbook exercises.
- **For later, once tenses feel solid:** AQ English — YouTube channel — run by two brothers focused on helping Urdu/Hindi speakers learn English, good for broader speaking/vocabulary practice once the tense foundation is in place.
- Optional spelling check: run daily log entries and project READMEs through Grammarly's free tier — catches spelling/grammar mistakes in real content already being produced, and surfaces personal repeat-mistake patterns over time.

Detailed Hour-by-Hour Reference

9:00 – 12:30 (Structured Learning)

Time	Activity
9:00 – 10:15	Watch videos (new material)
10:15 – 10:30	Short break (stretch, water, no phone scrolling)
10:30 – 11:45	Watch videos (continue) or switch topic if current one feels done for the day
11:45 – 12:30	Rewrite key concepts in your own words (not copy-paste — this is what makes it stick)

12:30 – 2:00 — Lunch break

2:00 – 4:00 (Continue Learning / Start Coding)

Time	Activity
2:00 – 3:00	Finish today's theory, or start hands-on practice tied to it
3:00 – 3:45	Hands-on practice / small exercises
3:45 – 4:00	Wrap up, jot down what's unclear (feeds tonight's log)

4:00 – 6:00 — Gym

Night (2-3 hrs, Coding/Projects + Practice)

Time	Activity
Hour 1	LeetCode: 1-2 Easy/Medium Python problems
Hour 2	SQL revision (30 min) + review today's confusing points (30 min)
Last 15-20 min	Daily log + GitHub commit if code was written today

On days without an active coding project (e.g. early Week 1), the night block is lighter — LeetCode + SQL + log is enough. Once project weeks start, this slot shifts toward real project coding.

Treat these as rough buckets in a clear order (learn → break → practice → gym → light coding/review → log), not a stopwatch — a video running 20 min long isn't a failure.

If You Fall Behind: Decide This Now, Not When You're Tired

The sequence (Math/Classical ML → Deep Learning → LLM/RAG → Backend → Docker → Cloud) is fixed — don't skip ahead to protect the calendar. But dates can flex. Use this rule of thumb instead of deciding in the moment:

Extend the current week (don't move on yet) if:

- You can't explain the core concept out loud, from memory, without notes (this is what your daily log's "what confused me" section is for — check it before moving on)
- Your project for the week is broken or you're faking the evaluation (e.g., only checking accuracy when you know precision/recall matters here)
- You're more than 2 days behind *within* the week — better to finish it properly than start the next one shaky

Push through and move on (don't extend) if:

- You understand the concept but just didn't finish every planned video/resource — extra resources are optional depth, not mandatory
- You're behind because of one bad day (illness, life event) but the core material landed fine
- The confusion is a minor edge case, not the central idea of the week

Hard rule, clarified per external review to remove a contradiction that existed here before: there are two different things that were being conflated — **protected rest** (your weekly buffer/rest day itself — never touched, never "borrowed") and **optional catch-up time** (extra depth/backup resources that were never mandatory in the first place). Never skip the *rest* part of a buffer day. If you need to extend a topic, that time comes from trimming optional resources or shifting dates — not from cannibalizing the rest you need to sustain a 12-week pace.

If you extend a week, borrow the time from, in this order:

1. Trim "optional depth" resources in the current or next 1-2 weeks (the ones marked as backups, not primary sources) — this is time that was never mandatory
2. Shift the whole remaining schedule back by the number of days needed — better a 13-week plan you actually complete, well-rested, than a 12-week plan with cracks in the foundation
3. **Never** repurpose an actual rest/buffer day as catch-up study time — that's the one thing that stays fixed no matter how behind you feel

Keep this short — 5-10 min, in your own words, no copy-paste from notes. Consistency matters more than polish. A rough 2-line entry on a packed day beats skipping it.

```
### Day [X] — [Date] — Week [X]: [Topic]

**What I learned today (2-3 sentences, from memory, no notes):**

**What confused me / not sure I fully understood:**

**What I built or coded today (even one line counts):**

**Tomorrow's focus:**
```

Where to keep it:

- Simplest: one running markdown file or Notion doc — low friction, you'll actually stick with it
- Portfolio bonus (optional): a `logs/` folder in your GitHub repo, or short daily posts on LinkedIn — do this only once the private habit is solid; don't let "make it public" become an excuse to skip a day

Why it's worth the 5-10 minutes:

- Forces retrieval practice — writing from memory (not while re-reading notes) reveals what you actually understood vs. what you just watched go by
- Gives you a paper trail to refresh Week 1 material once you're at Week 8 and it's gotten fuzzy
- The "what confused me" section is your built-in signal for what to revisit in the weekly buffer/review day
- Doubles as interview prep — six weeks of daily logs is a natural source for "tell me about a time you struggled with a concept and worked through it"

Week 7-8: LLM & GenAI Foundations (Sep 22–Oct 5, extended to 15 days total)

Goal: Move from "classical ML + basic DL" to "can build an LLM-powered application" — this is the core of what "AI Engineer" means in the current job market. This is likely the hardest conceptual stretch since Week 3's backprop — no prior week's knowledge transfers directly, it's genuinely new domain. Revised per external roadmap review: build fundamentals-first, framework-second, and correcting an earlier claim I made — LangChain is not inherently PyTorch-centric. PyTorch matters for

local model work/training, but is not required to build an API-based RAG application (most RAG work here calls a model provider's API, not a local PyTorch model).

Week 7 — LLM & RAG Foundations

Day	Focus	What to Do
Day 1	Intro to LLMs	How LLMs work, tokens, context window. **No need to go deep into Transformer math — high-level intuition is enough** (full depth is on the "Future Learning" list, post-employment)
Day 2	Hugging Face Basics	Load models, generate text, use pipelines
Day 3	Embeddings + Vector Database	Understand embeddings, use sentence-transformers, store/query them in a real vector DB — **prefer Chroma over FAISS** (easier to use, easier to save/load for a beginner)
Day 4	RAG Concept	How Retrieval-Augmented Generation works — chunking, retrieval, generation
Day 5	Build Simple RAG (Plain Python)	Document → chunk → embed → retrieve → generate, hand-built with an SDK, **no LangChain yet** — understand every step before automating it
Day 6	Improve RAG	Better chunking strategies, improved retrieval, first-pass manual evaluation (a handful of test questions, eyeballed)
Day 7	Buffer + Review	Clean code, push to GitHub, self-test the week's concepts from memory

Week 8 — Advanced RAG + Required Evaluation + Full Project

Day	Focus	What to Do
Day 1	Advanced RAG Concepts	Problems with basic RAG, better chunking strategies
Day 2	Query Improvement	Query rewriting, multi-query, HyDE — **keep this optional, only if there's extra time**

Day	Focus	What to Do
Day 3	Better Retrieval + LangChain	Hybrid search (vector + BM25 keyword baseline), re-ranking basics — bring in **LangChain** here, now that the manual version from Week 7 makes its automation legible
Day 4	**Build the Evaluation Set**	Write 30-50 real test questions with expected answers and source passages, including some the documents genuinely can't answer. This is required, not optional — it's the yardstick for everything that follows
Day 5	Build Full RAG Project	Start the main project (PDF Q&A or Chat with Documents), using LangChain + hybrid search + re-ranking
Day 6	Complete + Evaluate	Finish the RAG system, then run the Day 4 eval set against at least two configurations (e.g., basic RAG from Week 7 vs. this week's hybrid+re-ranked version). Report: retrieval accuracy, answer quality, citation behavior, **and abstention rate — how often the system correctly says "I don't know" when the answer isn't in the documents** (a RAG system that confidently makes things up is worse than one that admits uncertainty), plus response time/cost
Day 7	Add UI (Rough First Pass)	Streamlit or Gradio frontend — this is an early, rough deploy; the properly hardened, secured, containerized version comes in Weeks 9-11, so don't over-invest here
Day 8	Buffer + Polish	Clean code, strong README (include evaluation results, not just "it works"), record a short demo, self-test the whole 2-week block. **Also: a light 2-3 hour LangGraph overview** — not a build project, just enough conceptual + basic hands-on exposure to speak to it in an interview. Added after seeing a real job posting explicitly listing LangGraph/CrewAI/AutoGen as requirements alongside LangChain — this is the most LangChain-adjacent of the three (multi-agent/tool-using orchestration on top of what Week 8 already teaches), so it's the highest-value one to touch now rather than defer entirely

Resources:

- DeepLearning.AI — ChatGPT Prompt Engineering for Developers (free, ~1-2 hrs, still the best free starting point)
- DeepLearning.AI — LangChain: Chat with Your Data (free — use *after* building the plain-Python version first)

- DeepLearning.AI — Building and Evaluating Advanced RAG (free — this course's evaluation methodology is the required project requirement above, not optional)
- freeCodeCamp — YouTube channel (search "RAG from scratch freeCodeCamp" — long-form video covering query translation, routing, indexing, retrieval, generation in one sitting)

Note: LangChain has changed its package structure since some of these courses were recorded (it split into langchain-core/langchain-community/langchain). You'll cross-reference the current LangChain docs — that's normal and expected, not a sign you're doing something wrong.

Awareness only, no dedicated time: Large Multimodal Models (LMMs) — LLMs that handle images/audio, not just text — are a natural "what's next" after this foundation. Worth knowing they exist and roughly what they do (useful in interviews), not worth a build project at this stage.

Week 9: Strengthen the RAG Backend (Oct 6–12)

Goal: Basic FastAPI skills were already built in Week 5 on your HR/Churn project — this week applies them to make your Week 7-8 RAG app genuinely robust, not just working on a lucky demo run.

Topics: Input validation specific to LLM apps, failure handling (what happens when the API times out, or retrieval finds nothing relevant), tests for the RAG pipeline specifically, basic access control

Project: Wrap your Week 7-8 RAG chatbot in a FastAPI backend with:

- Proper input validation and clear error responses (not stack traces) for malformed requests
- Failure handling for LLM API timeouts, rate limits, and empty/irrelevant retrieval results
- A few real tests: does the retrieval step behave correctly on a known query? Does the app correctly abstain when it should? Does it reject bad input?
- Basic access control appropriate for a demo (not enterprise-grade, just not wide open)

Resources:

- FastAPI — Official Tutorial (revisit specifically the validation and error-handling sections)

Day 7: Buffer + review

Week 10: Containerization (Oct 13–19)

Goal: Package your application so it runs anywhere — a baseline expectation for "engineer" roles.

Topics: Docker basics, Dockerfile, docker-compose, containerizing a FastAPI app

Resources:

- Docker — Official Get Started Guide
- Docker Docs — FastAPI containerization samples

Project: Containerize your FastAPI + RAG project from Weeks 8-9 with a Dockerfile, and confirm it runs identically in a fresh container.

Day 7: Buffer + review

Week 11: Cloud Deployment Basics (Oct 20–26)

Goal: Deploy beyond your own machine — this is what turns a "project" into a "live, demoable product."

Topics: Basic cloud deployment (AWS EC2 free tier or Google Cloud Run — pick one, don't split focus), environment variables, basic monitoring/logging

Resources:

- Search "[deploy FastAPI Docker AWS EC2 free tier]" or "[deploy FastAPI Docker Google Cloud Run]" for a current step-by-step — cloud UIs change often enough that a fixed tutorial link ages fast; the official free-tier docs below are the most reliable starting point
- AWS Free Tier | Google Cloud Run

Awareness only, no dedicated time: dedicated model-serving tools (TorchServe, BentoML) exist for scaling model deployment beyond a single FastAPI+Docker setup. Worth knowing the names and one-line purpose for interviews — "the next level up from what I've built" — not worth building with at this stage.

Security & reliability checklist — fold into Weeks 10-11, not a separate study phase (per external roadmap review):

- Automated tests + a simple check that runs when code changes
- Logging that doesn't unnecessarily retain sensitive document content
- Upload/input limits, proper secret handling, access control appropriate to a demo (not enterprise-grade)
- Basic prompt-injection tests: retrieved documents are evidence for the model to use, not instructions for it to obey — verify your app doesn't blindly execute something a malicious document tells it to do
- Persistence for your vector index, and a genuinely tested restart/redeployment (don't just assume it'll come back up correctly)

- Usage limits and cloud/API billing alerts — **"free tier" is not a guarantee of zero charges, and alerts alone don't cap spending; set actual limits where the platform allows it**
- For any public demo, use public or synthetic documents only — never real/sensitive data in something reachable by strangers
- **Explicitly out of scope for this phase:** Kubernetes, distributed training, complex multi-agent systems, extensive fine-tuning — these are genuinely valuable but not needed to ship a working, honest demo

Day 7: Buffer + review

Week 12: Final Evaluation, Documentation & Applications (Oct 27–Nov 2)

Goal: Two polished portfolio anchors — one ML project (Week 5's deployed HR/Churn service) and one evaluated, deployed LLM application (the RAG project) — not three half-finished efforts. The CNN project from Week 4 stays a supporting learning project, not a third polishing effort.

- Final evaluation pass on the RAG project using your Week 7-8 evaluation set — confirm the numbers are still accurate after Weeks 9-11's changes
- Documentation: a README explaining the architecture (embeddings → vector store → retrieval → generation), how to reproduce your results, and what the evaluation showed, including where it fails
- Record a short demo video of both projects (RAG app + deployed HR/Churn service) — many recruiters/interviewers will watch a 2-3 minute demo before reading code
- Update GitHub, LinkedIn, and resume specifically around these two projects — this is what you'll walk interviewers through in depth
- Interview practice — be ready to explain your design choices out loud, not just show that it works
- Start applying with "AI Engineer" as a target title alongside "ML Engineer" and "Data Scientist" — cast a slightly wider net since exact title definitions vary a lot by company

What "Ready" Should Mean

Per external roadmap review: treat 12 weeks as an ambitious foundation-and-portfolio sprint, not a promise of employment. Completing the timetable alone doesn't establish readiness — being able to do the following, independently, does:

- Build and evaluate a leakage-safe ML pipeline

- Explain your neural-network training and diagnose overfitting from a training/validation curve
- Build a RAG system and demonstrate — with actual numbers — when and how it fails
- Serve, test, containerize, and deploy an application
- Explain your design choices and reproduce your results from the README alone, without needing to re-explain verbally

  If any of these feel shaky by Week 12, that's a real, useful signal — not a failure of the roadmap, just an honest checkpoint of where more time is actually needed before or during applications.

Where You Actually Stand (updated each session)

- Week 1: ✅ Complete
- Week 2: ✅ Complete (with significantly more depth than originally scoped — 6 algorithms, full pipelines, feature engineering with evidence-based conclusions, K-Means, PCA)
- Week 3: ✅ Complete
- Week 4: ✅ Complete (CNN + ablation-tested regularization, VGG19 transfer learning with real evidence comparison, light RNN/LSTM overview)
- Week 5: ✅ Complete — and then some. Churn ANN deployed as a real product: clean module structure, fixed data-leakage bug, Git/GitHub, FastAPI + Pydantic validation (verified), 3 passing pytest tests, **and live on Render** (churn-api-hk05.onrender.com) — meaning a real chunk of Weeks 10-11's cloud deployment scope is already done
- Week 6 next: portfolio polish, applications begin (per the plan, this is also the point to start applying for ML Intern/Data Analyst/Junior Data Scientist roles)

v4 Revision Note (external roadmap review)

A detailed external review of this roadmap (covering pp. 4-12) led to real structural changes, not just additions:

- Week 4: dropped the redundant second full PyTorch rebuild (already done once in Week 3); added transfer learning + error analysis as the actual project deliverable instead
- Weeks 5-6: restructured to bring basic software engineering (Git, FastAPI, tests, clean code) forward, deployed on your *existing* HR/Churn project instead of starting a new one — no longer waiting until Week 9 to find out if a notebook works as a real application
- Weeks 7-8: build RAG fundamentals with plain Python + an SDK *before* LangChain, not framework-first; added a required RAG evaluation methodology (30-50 question eval set, retrieval/answer/citation/cost metrics, comparing configurations) instead of an untested "impressive-looking" demo; corrected an earlier claim that LangChain requires PyTorch (it

doesn't — most RAG work here is API-based); added a light BM25 baseline for comparison, not a full traditional-NLP detour
- Week 9: refocused specifically on hardening the RAG backend (validation, failure handling, tests, access control), since general FastAPI basics moved to Week 5
- Weeks 10-11: added a concrete security/reliability checklist (prompt-injection awareness, secret handling, billing alerts, tested restarts) folded into existing deployment work, not a new phase
- Daily structure: protected the 2:00-4:00 block specifically for hands-on implementation, not more videos — target at least half of daily study time actually building/debugging/testing
- Fixed a real contradiction in the "if you fall behind" rules: protected rest days are never borrowed from; only optional/backup study time is

Future Learning — Post-Employment, Not Part of This Roadmap

Compared against a research-engineer-oriented roadmap (roadmap.sh). These are genuinely good topics, but for a different career track (Research Engineer/Applied Scientist) than this plan's goal (job-ready AI Engineer in 12 weeks). Revisit these once employed, not before.

- Traditional NLP (BoW, TF-IDF) — largely superseded by modern LLM approaches for this path
- RNN/LSTM/Seq2Seq/Attention from scratch — historical foundation for Transformers, not required to use them via Hugging Face/LangChain
- Transformer architecture deep math (self-attention, positional encoding, from scratch) — a few hours of *intuition* is folded into Week 7-8; full depth is research territory
- LoRA/QLoRA fine-tuning — real, in-demand skill, but needs GPU access + real time investment; advanced add-on
- Time Series + Forecasting — job-dependent, not core to this path
- Advanced Computer Vision (Object Detection, Segmentation) — beyond Week 4's CNN/transfer learning scope
- Multimodal Models (LMMs) — awareness only added in Week 7-8; full depth later
- Advanced Model Serving (TorchServe, BentoML) — awareness only added in Week 11; full depth later
- **CrewAI, AutoGen** — multi-agent orchestration frameworks, same family as LangGraph (which moved into Week 8 Day 8 after appearing as a real job requirement). These two stay deferred since they'd be redundant with LangGraph's conceptual coverage — revisit if a specific role asks for one by name
- Research Engineer track, paper reading — different career goal entirely

Quick-Reference Link Library (all in one place)

Purpose	Resource	Link
Linear Algebra	3Blue1Brown	https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab
Calculus	3Blue1Brown	https://www.youtube.com/playlist?list=PLZHQObOWTQDMsr9K-rj53DwVRMYO3t5Yr
Statistics	StatQuest	https://statquest.org
ML theory (primary, beginner-friendly)	Andrew Ng — ML Specialization (Coursera, free audit)	https://www.coursera.org/specializations/machine-learning-introduction
ML theory (deep, math-heavy — optional 2nd pass)	Stanford CS229, Autumn 2018	https://www.youtube.com/playlist?list=PLoROMvodv4rMiGQp3WXShtMGgzqpfVfbU
ML practice, hands-on micro-courses	Kaggle Learn (Intro/Intermediate ML, Feature Engineering)	https://www.kaggle.com/learn
ML theory (practical, video, pick as primary)	Krish Naik	https://www.youtube.com/@krishnaik06
ML theory (practical, structured day-by-day, pick as primary)	CampusX — 100 Days of ML	https://www.youtube.com/@campusx-official
ML projects, end-to-end (backup source)	Codebasics	https://www.youtube.com/@codebasics

Purpose	Resource	Link
PyTorch, papers-from-scratch	Aladdin Persson	https://www.youtube.com/@AladdinPersson
Neural net fundamentals	deeplizard	https://www.youtube.com/@deeplizard
Deep Learning, full course	fast.ai	https://course.fast.ai/
LLMs / Transformers	Hugging Face Course	https://huggingface.co/learn
Deployment (Streamlit)	Streamlit Docs	https://docs.streamlit.io
Deployment (Gradio)	Gradio Docs	https://www.gradio.app/docs
Datasets, notebooks, GPUs	Kaggle	https://www.kaggle.com
Kaggle's own free courses	Kaggle Learn	https://www.kaggle.com/learn
Portfolio hosting	GitHub	https://github.com
Prompt engineering	DeepLearning.AI (ChatGPT Prompt Engineering)	https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/
RAG / LangChain basics	DeepLearning.AI (LangChain: Chat with Your Data)	https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/
RAG evaluation (advanced)	DeepLearning.AI (Building and Evaluating Advanced RAG)	https://www.deeplearning.ai/short-courses/building-evaluating-advanced-rag/

Purpose	Resource	Link
Backend APIs	FastAPI official tutorial	https://fastapi.tiangolo.com/tutorial/
Containerization	Docker — Get Started	https://docs.docker.com/get-started/
Cloud deployment	AWS Free Tier	https://aws.amazon.com/free/
Cloud deployment (alt)	Google Cloud Run	https://cloud.google.com/run

Note: Krish Naik, Codebasics, CampusX, Aladdin Persson, and deeplizard are linked to their channel homepages rather than a specific playlist — their playlists get reorganized over time, so searching the channel for the exact topic (e.g. "Krish Naik Random Forest") is more reliable than a fixed deep link.

Caution: where two sources are listed as "pick ONE," don't use both for the same topic. Resource-hopping (watching the same concept explained by 3 different YouTubers) feels productive but usually means less retention per hour, not more. Pick a primary source per topic, and only reach for a second explanation if the first genuinely didn't click.

What makes this a 10/10 instead of a 7/10

1. No relearning material you've already mastered
2. Realistic pacing — deep learning gets 2 weeks, not 1
3. A rest/buffer day every week so falling behind once doesn't sink the plan
4. Spaced review so concepts survive past the week you learned them
5. ~6 weeks instead of 30 days — still fast, but survivable and retainable
6. Applications start *during* the final week, not after everything is done